

Git for Developers

Closing — Pitfalls, Cheat Sheet & Resources

Common Mistakes, Quick Reference, and What to Learn Next

Closing — Agenda

- Common pitfalls and how to avoid them
- Git command cheat sheet
- Recommended resources and next steps
- Q&A

Part 1 — Common Pitfalls

Pitfall 1 — Committing Secrets

```
# Someone accidentally committed an API key  
git commit -m "add env config"  
# → .env file with SECRET_KEY=sk-live-abc123 is now in history  
  
# Even after deleting the file, it's still in history!  
git rm .env  
git commit -m "remove env"  
# The secret is still in git log -p HEAD~1
```

How to avoid it

```
# Add secrets to .gitignore BEFORE the first commit
```

Pitfall 2 – Committing to the Wrong Branch

You committed 3 things directly to main by accident

```
git log --oneline -3
```

a3f7c21 feat: payment integration

b2e1d40 feat: shipping calculator

c9a8b12 feat: tax module

Move those commits to a feature branch

```
git branch feature/ecommerce
```

create branch at current HEAD

```
git reset --hard origin/main
```

move main back to origin

```
git checkout feature/ecommerce
```

the 3 commits are still here

Pitfall 3 — Force-Pushing Shared Branches

Dangerous — overwrites others' work

```
git push --force origin main
```

Safer — aborts if someone pushed since your last fetch

```
git push --force-with-lease origin main
```

Safest — just don't force-push shared branches

Rewrite history only on your local, private feature branches

Pitfall 4 — Merge vs. Rebase Confusion

```
# After git pull --rebase, your commits were rewritten  
# (new SHAs) – don't panic!
```

```
git pull --rebase
```

```
# CONFLICT (content): Merge conflict in src/auth.js
```

```
# Resolve, then:
```

```
git add src/auth.js
```

```
git rebase --continue    # NOT git commit!
```

```
# Abort if you change your mind
```

```
git rebase --abort
```

Pitfall 5 — Losing Work in `git reset --hard`

```
# You ran: git reset --hard HEAD~5 and lost 5 commits  
# Recovery via reflog  
git reflog  
# a3f7c21 HEAD@{6}: commit: feat: last important commit  
  
git reset --hard a3f7c21    # you're back!
```

The reflog saves you from almost everything — except `git clean -fdx` (irreversible).

Pitfall 6 — Giant Commits / PRs



One commit: "Implement entire e-commerce module"

→ impossible to review, impossible to bisect, impossible to revert cleanly



Many small commits:

feat: add product catalog data model

feat: add product listing API endpoint

feat: add cart service

feat: add checkout flow

feat: add payment gateway integration

Part 2 — Git Cheat Sheet

Setup & Configuration

```
git config --global user.name "Jane Smith"  
git config --global user.email "jane@example.com"  
git config --global core.editor "code --wait"  
git config --global init.defaultBranch main  
git config --list
```

Creating & Cloning

```
git init                                # initialize a new repository
git clone <url>                          # clone a remote repository
git clone --depth 1 <url>                # shallow clone (CI-friendly)
git clone --recurse-submodules <url>    # clone with submodules
```

Daily Workflow

<code>git status</code>	<i># what's changed?</i>
<code>git add <file></code>	<i># stage a file</i>
<code>git add .</code>	<i># stage everything</i>
<code>git add -p</code>	<i># stage interactively (hunk by hunk)</i>
<code>git commit -m "feat: ..."</code>	<i># commit with message</i>
<code>git commit --amend</code>	<i># fix last commit (before push)</i>
<code>git push</code>	<i># push to remote</i>
<code>git pull --rebase</code>	<i># pull and rebase</i>

Branching

<code>git branch</code>	<i># list local branches</i>
<code>git branch -a</code>	<i># list all branches (+ remote)</i>
<code>git checkout -b feature/x</code>	<i># create and switch</i>
<code>git switch -c feature/x</code>	<i># modern equivalent</i>
<code>git checkout main</code>	<i># switch to main</i>
<code>git merge feature/x</code>	<i># merge into current branch</i>
<code>git merge --no-ff feature/x</code>	<i># merge with merge commit</i>
<code>git branch -d feature/x</code>	<i># delete merged branch</i>
<code>git branch -D feature/x</code>	<i># force-delete</i>

History & Inspection

```
git log --oneline --graph --all      # visual branch history
git log --grep="auth"                # search commit messages
git diff                             # unstaged changes
git diff --staged                    # staged changes
git show <hash>                      # show a commit
git blame <file>                     # annotate file with commit info
```

Undoing

```
git restore <file>           # discard unstaged changes
git restore --staged <file>   # unstage a file
git reset --soft HEAD~1       # undo commit, keep staged
git reset --hard HEAD~1       # undo commit, discard changes
git revert <hash>              # safe undo (creates new commit)
git reflog                    # find lost commits
```


Advanced

<code>git rebase main</code>	<i># update branch with main's changes</i>
<code>git rebase -i HEAD~4</code>	<i># interactive rebase</i>
<code>git cherry-pick <hash></code>	<i># apply one commit to current branch</i>
<code>git stash</code>	<i># save WIP</i>
<code>git stash pop</code>	<i># restore WIP</i>
<code>git tag -a v1.0.0 -m "..."</code>	<i># create annotated tag</i>
<code>git bisect start/good/bad</code>	<i># binary search for a bug</i>
<code>git clean -n</code>	<i># preview removal of untracked files</i>
<code>git clean -fd</code>	<i># remove untracked files + dirs</i>

Part 3 — Resources & Next Steps

Essential Reading

Resource	URL / Description
Pro Git (book)	git-scm.com/book — free online, comprehensive
Official Git docs	git-scm.com/docs — every command explained
Conventional Commits	conventionalcommits.org
Gitflow	nvie.com/posts/a-successful-git-branching-model

Interactive Learning

Platform	Content
Learn Git Branching	learngitbranching.js.org — visual, interactive branching exercises
GitHub Skills	skills.github.com — guided repo-based courses
Katacoda / Killercodea	Hands-on terminal scenarios
Oh My Git!	Game-based Git learning

Tools to Explore Next

Tool	Purpose
Husky	Git hooks for Node.js projects
commitlint	Enforce Conventional Commits in CI
git-filter-repo	History rewriting (safer than <code>filter-branch</code>)
BFG Repo Cleaner	Remove large files or secrets from history
Scalar / VFS for Git	Scale Git for very large repos
Nx / Turborepo	Monorepo tooling
lazvgit	Terminal UI for Git

Thank You!

Key Takeaways

- **Commit small, commit often** — atomic commits make everything easier
- **Branch freely** — they're cheap and safe
- **Never rewrite shared history** — use `git revert` on public branches
- **The reflog is your safety net** — most "lost" work is recoverable
- **Automate with hooks and CI** — let the machine enforce quality

Q&A

"The most powerful Git skill is knowing when NOT to use a complex command — and choosing the simple thing that works."

Keep Practising

```
# Today's homework  
git init my-practice-repo  
cd my-practice-repo  
git checkout -b feature/experiment  
# ... make some commits, try a rebase, resolve a conflict ...  
git log --oneline --graph --all
```

Good luck, and happy committing!